Project Overview & Current Progress Assessment Report

Project Name: Solar Rooftop Suitability Assessment Tool
Event / Target: Smart India Hackathon (SIH) 2026
Report Type: Technical Progress, Architecture & Readiness Assessment
Date: October 19, 2026

1. Project Overview

The **Solar Rooftop Suitability Assessment Tool** is an interactive, web-based decision-support system designed to empower property owners, clean-energy planners, and residential/commercial solar consumers.

Its primary goal is to eliminate uncertainty prior to capital expenditure on photovoltaic installations by answering three essential questions:

1. **Is the rooftop suitable** for solar panel deployment in terms of area, structural layout, and solar exposure?
2. **What is the true usable rooftop area** after subtracting physical obstructions, shade-casting structures, and required safety setbacks?
3. **What is the optimal solar array capacity and layout** required to offset the user's actual electricity consumption without oversizing or escalating installation costs?

The platform guides users through an intuitive, five-stage analytical journey—bridging map data, geometric obstacle placement, solar insolation estimates, and consumption-driven panel placement.

2. Problem Statement

Rooftop solar adoption faces significant barriers at the pre-installation phase:

- **Lack of Feasibility Knowledge:** Property owners often do not know whether their roof orientation, slope, and shading profile make solar financially or physically viable.
- **Complex Usable Area Calculation:** Manual measurement of irregular roofs is error-prone. A 1,000 sq.ft roof cannot accommodate 1,000 sq.ft of panels; parapets, walkways, and setbacks

diminish the installable surface.

- **Impact of Obstacles:** Water tanks, HVAC/AC condenser units, staircase headrooms, skylights, chimneys, and nearby trees cast shadows and physically block panel installation. Neglecting these causes severe system underperformance and hot-spot degradation.
- **Risk of System Oversizing:** Installers frequently propose maximum-capacity installations rather than demand-matched setups, leading to inflated upfront costs, extended payback periods, and unutilized excess generation.
- **Absence of Intelligent Layout Planning:** Placing panels without respecting keep-out buffers, maintenance walkways, and local solar irradiance zones results in sub-optimal system yield and code violations.

3. Our Proposed Solution

The project addresses these challenges through a modular, five-stage workflow:

Phase 1: Location (`/assessment`)

- **Objective:** Pinpoint the target structure on high-resolution maps.
- **Mechanism:** Integrates Nominatim address geocoding, device geolocation, reverse geocoding, and OpenStreetMap (OSM) Overpass API queries to detect building footprints within a 120-meter radius. If no OSM polygon exists, users can manually outline the structure.

Phase 2: Rooftop & Obstacle Annotation (`/assessment/rooftop`)

- **Objective:** Establish the precise boundary and catalog all obstructions.
- **Mechanism:** Accepts the footprint from Phase 1. Users can refine boundaries and mark obstacles (water tanks, AC units, trees, vents, etc.) as points, rectangles, or polygons with customizable height and safety clearance buffers.

Phase 3: Solar Resource & Shading Analysis (`/assessment/solar`)

- **Objective:** Map spatial solar potential across the rooftop surface.
- **Mechanism:** Computes sun positions and runs an astronomical empirical solar radiation model (Hottel / Duffie & Beckman) along with obstacle shading projection to classify zones into HIGH, MEDIUM, or LOW potential. The user then selects the active solar installation zone.

Phase 4: Demand-Driven Optimization (`/assessment/optimization`)

- **Objective:** Size and arrange photovoltaic arrays to match electricity bills.
- **Mechanism:** Prompts for monthly power consumption (or electricity bill in ₹). Derives required annual kilowatt-hours (kWh) and required module count. Generates candidate array layouts (e.g., *Consumption-Optimized*, *Maximum Capacity*), fitting modules in high-insolation zones while respecting setbacks and keep-out buffers.

Phase 5: Results & Energy Simulation (`/assessment/energy`)

- **Objective:** Deliver finalized system specifications, energy comparisons, and exportable assets.
- **Mechanism:** Summarizes system capacity (kWp), module count, occupied area, annual yield, self-supply ratio, and month-by-month generation vs. consumption balance. Enables one-click export of structured JSON and GeoJSON datasets.

4. End-to-End System Flow & Architecture

Internal Implementation Highlights:

1. **User Inputs:** Search query or map pin, building selection, obstacle geometries/heights, solar zone selection, and monthly bill amount (₹) or consumption (kWh).
2. **Location & Imagery:** OpenStreetMap tiles rendered via Leaflet/React-Leaflet; Nominatim for geocoding; Overpass API for footprints.
3. **Usable Area Deduction:** Vector polygon arithmetic subtracts obstacle footprints and perimeter clearance buffers from the gross roof area.
4. **Solar Calculation:** Centroid-based empirical model calculating extraterrestrial radiation (H_0 H0), clearness index (K_t Kt), and peak sun hours without hard external dependencies.
5. **Panel Placement:** Grid-packing algorithm aligned to orientation/tilt constraints, verifying that panel bounds do not intersect obstacle safety buffers.
6. **State Persistence:** Entire pipeline is client-persisted in `sessionStorage` with validation routines on route entry.

5. Current Progress Assessment

Feature or Phase	Current Status	Evidence or Explanation
Phase 1: Location	**Implemented**	Working Nominatim search, GPS geolocation, map click reverse geocoding, OSM Overpass building footprint discovery, manual building drawing, and site confirmation modal (`AssessmentWorkspace.tsx` , `AssessmentContext.tsx`).
Phase 2: Rooftop	**Implemented**	Boundary drawing/redrawing, multi-geometry obstacle creation (points, rectangles, polygons), clearance zones, rooftop section division, unsaved change warnings (`RooftopPage.tsx` , `RooftopContext.tsx`).
Phase 3: Solar	**Implemented**	Physics-based solar resource computation (`solarResourceService.ts`), shading evaluation (`shadingService.ts`), zone segmentation, solar timeline, and solar installation area selection (`SolarPage.tsx` , `SolarContext.tsx`).
Phase 4: Optimization	**Implemented**	Layout generation engine supporting multiple objectives (`max_capacity` , `consumption_optimized`), panel specs library, obstacle collision checks, undo/redo, manual panel deletion/restore (`OptimizationPage.tsx` , `OptimizationContext.tsx` , `panelOptimizationService.ts`).
Electricity Bill Image Input (OCR)	**Partially Implemented**	**Service exists; UI missing.** `billExtractionService.ts` integrates Tesseract.js with regex parsers for annual/monthly kWh extraction (validated in `tests/bill-extraction.test.ts`). However, `ConsumptionModal.tsx` currently only provides a numeric ₹ input field with presets without a file upload selector.
Obstacle Detection (Automated/AI)	**Pending / Stubbed**	`RooftopContext.tsx` contains state fields (`aiDetectionStatus`) and action stubs, but no automated computer-vision or satellite detection model is connected. All obstacles must currently be annotated manually by the user.
Panel Requirement Calculation	**Implemented**	Formula converts bill ₹ to kWh using tariff rate (₹8/kWh default), computes required kWp using local peak sun hours, calculates target module count, and caps at roof physical maximum (`consumptionUtils.ts` , `panelOptimizationService.ts`).
Results and Reporting	**Implemented**	`EnergyPage.tsx` displays array metrics, monthly generation vs. consumption balance, surplus/shortfall, self-supply ratio, and functional JSON & GeoJSON export buttons (`dataExportService.ts`).

6. What We Have Achieved So Far

- **End-to-End Five-Phase Routing:** Complete React Router v7 workflow with route-guard validation to prevent skipped steps.
- **Dual Building Acquisition:** Automated GIS footprint fetching from OpenStreetMap alongside a fallback manual polygon drawing tool.
- **Granular Obstacle Modeling:** Ability to define point obstacles (vents), rectangular units (HVAC), or irregular polygons (headrooms/trees) with clearance buffers.
- **Physics-Based Insolation Engine:** Fully functional local solar modeling using Hottel/Duffie-Beckman clear-sky equations tailored by latitude.
- **Consumption-Matched Array Sizing:** Dynamic panel sizing based on electricity bills rather than blind area saturation.
- **Interactive Optimization Workspace:** Interactive module manipulation including deletion, restoration, locking, and side-by-side layout comparisons.
- **Comprehensive Exporting:** Full project state serialization to standard JSON and geospatial GeoJSON.
- **OCR Service Core:** Completed Tesseract.js extraction pipeline capable of parsing utility bills into consumption profiles.

7. What Remains (Prioritized Deficits)

1. **Connect Bill OCR to Optimization UI (High Priority):**
   Add a drag-and-drop file upload zone in `ConsumptionModal.tsx` to pass uploaded bill images directly to `extractConsumptionFromImage()`.
2. **End-to-End Runtime Validation (High Priority):**
   Execute full browser walkthroughs across diverse geographical coordinates to verify Overpass API timeouts, Leaflet polygon rendering performance, and edge-case geometries.
3. **Automated AI Obstacle Detection (Medium Priority):**
   Replace the `aiDetectionStatus` stub with an actual lightweight CV model (or serverless segmentation endpoint) for automated obstacle inference from aerial tiles.
4. **Live Weather / Solar API Integration (Medium Priority):**
   Augment the local empirical radiation model with live API feeds (e.g., NREL PVWatts, NASA POWER, or PVGIS) with fallback to the local model.
5. **Configurable Tariff Rates (Medium Priority):**
   Allow users to customize the electricity tariff rate (currently hardcoded to ₹8.0/kWh) according to their state/discom tariff schedules.

8. Our Exact Current Position

- **Development Stage:** Late-stage functional prototype / Pre-MVP.
- **What is Available:** A working end-to-end web application covering location discovery, rooftop markup, solar zoning, bill-based panel sizing, layout customization, and reporting.
- **What is Being Completed:** Unifying the already-coded bill OCR service with the UI modal, and validating live map/data service resilience.
- **Next Logical Development Step:**
  Integrate the file upload input in `ConsumptionModal.tsx` to allow bill uploads alongside manual entry, followed by a full team QA pass over sample properties.

9. Technology Stack Summary (Verified in Codebase)

- **Core Framework:** React 19, TypeScript (~6.0), Vite 8
- **Routing:** React Router DOM (v7.18)
- **GIS & Mapping:** Leaflet 1.9, React-Leaflet 5.0, OpenStreetMap Tiles, Nominatim API (Geocoding), Overpass API (OSM Footprints)
- **Computer Vision / OCR:** Tesseract.js (v7.0)
- **UI Animation & Styling:** Framer Motion (v13), CSS Modules, SVG vector graphics
- **Persistence & Architecture:** Client-side `sessionStorage` state persistence, Context + Reducer pattern, zero backend/database dependencies.

10. Executive Summary for Team Meetings (1–2 Minute Script)

> *"Our Solar Rooftop Suitability Assessment Tool for SIH 2026 is in a late-stage prototype phase with all five core phases implemented and connected.*
>
> *The problem we solve is simple: property owners want solar, but they don't know if their roof is suitable, how much space is lost to obstacles like tanks and AC units, or how many panels they actually need. Installers often oversize systems to maximize contract size.*
>
> *Our system guides users through five steps: selecting their property via OpenStreetMap footprints, marking obstacles with clearance buffers, evaluating solar exposure zones, entering their monthly electricity bill to place the exact number of panels needed, and viewing a full energy report with GeoJSON export.*
>
> *Where we stand right now: The entire pipeline from map selection to layout optimization is coded and functional in client state. Crucially, the OCR service to read electricity bills using Tesseract.js is already written and tested—we just need to expose the file upload button on the optimization modal. Our immediate next step is adding that UI toggle and doing a full end-to-end verification run."*